

Zachary Kosto

Professor Wilson

CS320

13 October 2025

Throughout this project, I developed and tested three main features for a mobile application: the contact service, task service, and appointment service. My unit testing approach for each focused on verifying that every class and service met the specific requirements outlined in the customer's request. For example, in the *Contact* and *Task* classes, I created JUnit tests to ensure that each field followed the required length and null constraints. In the *Appointment* class, I also included logic that verified the appointment date was not in the past, which directly aligned with the given requirement. Each test I wrote was designed to confirm that the classes behaved as expected under both valid and invalid conditions.

My approach was closely aligned with the software requirements because I based every test on the individual constraints defined for each field. For instance, the phone number test in *ContactTest.java* checked for exactly 10 digits, while the *AppointmentTest.java* verified that the date field returned an error if set before the current day. These tests ensured that data validation matched the exact limits the customer wanted. I also performed coverage tests to make sure that the results reached at least 80%, which showed that most lines of code were executed during testing. This level of coverage gave me confidence that the JUnit tests were effective and reliable.

Writing the JUnit tests was a positive experience because it helped me see how testing can uncover small issues early in development. I made sure my code was technically sound by using assertions like `assertEquals`, `assertTrue`, and `assertThrows` to validate behavior in different scenarios. For example, in my *TaskServiceTest*, I used `assertThrows(IllegalArgumentException.class, () = service.addTask(null));` to make sure invalid input was properly handled. I also kept the tests efficient by avoiding unnecessary repetition and grouping related checks within the same method when possible. This kept the code clean and easy to maintain while still testing all the necessary functionality.

For this project, the main software testing technique I used was unit testing, which focuses on verifying small, isolated pieces of code such as individual methods or classes. This technique made it easier to find logic errors before they could affect other parts of the application. I also used boundary testing by checking edge cases like maximum string lengths and invalid dates. These techniques helped ensure that the application behaved correctly under different inputs and conditions.

Some testing techniques I did not use were integration testing and system testing. Integration testing would have checked how the services worked together as a complete system, while system testing would verify the application as a whole from start to finish. In this project, those methods were not necessary because the focus was only on the back-end logic, but they would be valuable in larger projects that include multiple modules or user interfaces.

In real-world software development, different testing techniques serve unique purposes. Unit testing is ideal during early development, while integration and system testing are used before deployment to catch issues between components. Boundary and regression testing also

play important roles in maintaining code quality over time. Understanding when to use each technique helps make testing strategies more effective.

Throughout this project, I made sure to approach testing with caution and attention to detail. Even though the services were small, I recognized how one overlooked input or validation could cause larger issues later on. For example, if I had not checked for invalid dates in the appointment class, users could schedule events in the past, which would lead to functionality errors. Appreciating those small details helped me stay focused and careful.

I also tried to limit bias while reviewing my code. Since I wrote both the program and the tests, it was easy to assume everything was correct, so I made a point to test unexpected scenarios and invalid inputs to prove that the code could handle failure correctly. As a developer, it's important to step back and think like a tester rather than assume your code works. This mindset helps reduce overconfidence and leads to better quality results.

Finally, this project reminded me how important discipline and commitment to quality are in software development. Cutting corners or skipping tests might save time short-term, but it creates technical debt that becomes harder to fix later. I plan to avoid that by continuing to write thorough tests, documenting code clearly, and reviewing logic carefully before moving on. Taking the time to do things right from the start prevents problems and helps maintain a strong standard of work.